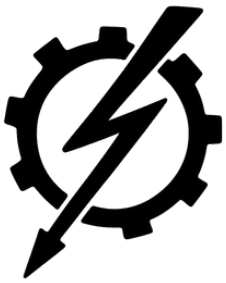
TWORZENIE APLIKACJI WEBOWYCH



```
|||> cat ${CURRENT_LAB}
```

```
<!DOCTYPE html>
<html>
<head>
```

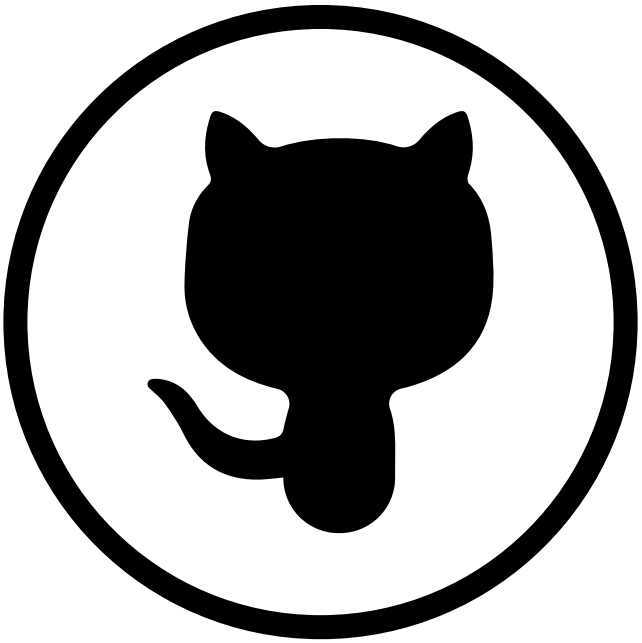
<title> WEB APPS </title>

```
</head>
<body>
```

<h1> LAB 2 </h1>

Git - Rebase, Code Review

```
</body>
</html>
```



```
|||> echo “by ${AUTHOR}”
```

by Mateusz Węglewski

Task 2.0.

Przygotuj środowisko.

1. **Wejdź na *https://shell.cloud.google.com* i zaloguj się kontem Google.**
2. **Skonfiguruj swoją tożsamość w gicie: (może wystąpić konieczność powtórzenia tego)**

```
git config --global user.name "Imię Nazwisko"
git config --global user.email "twoj@email.com"
```

Task 2.1.

Sklonuj repozytorium i wejdź do środka.

```
mkdir lab2 && cd lab2
git clone https://github.com/eficode-academy/git-katas.git
cd git-katas
```

Task 2.2.

Wykonaj odpowiednie kroki i odpowiedz na pytania.

1. **Wykonaj komendy.**

```
cd rebase-branch
source setup.sh
```

2. **Wykonaj *git log --oneline --graph --all*. Ile jest gałęzi? Na którym commicie się rozchodzą?**

_____

_____

3. **Przełącz się na gałąź *uppercase*. Zapisz hash commita "Change greeting to uppercase:"** _____
4. **Wykonaj *git rebase master*. Ponownie wykonaj *git log --oneline --graph --all*. Co się zmieniło w grafie?**

_____

_____

5. **Porównaj hash commita "Change greeting to uppercase" z tym z pkt 3. Czy jest taki sam? Dlaczego?**

_____

_____

Task 2.3.

Wykonaj odpowiednie kroki i odpowiedz na pytania.

1. **Wykonaj komendy.**

```
cd ../../merge-conflict
source setup.sh
git switch merge-conflict-branch1
```

2. **Przenieś gałąź na *master* poleceniem *rebase*. Co się stało?**

_____

3. Na jakiej gałęzi jesteś?

_____

4. Otwórz *file.txt*. Co jest pod <<<<<<< HEAD: zmiana z *master* czy z twojej gałęzi?

_____

5. Rozwiąż konflikt tak, żeby zostały obie linie. Dokończ rebase zgodnie z podpowiedzią z *git status*. Jakich poleceń użyłeś? Czym różni się to od kończenia merge?

_____

_____

6. Jakim poleceniem przerwiesz rebase i wrócisz do stanu sprzed niego?

_____

Task 2.4.

Kolega otworzył Pull Request. Zrób review: znajdź co najmniej 4 problemy, napisz do każdego krótki komentarz i podejmij decyzję: *Approve*, *Comment* albo *Request changes*.

```
PR #12: fix → main
 Branch: test67
 Description: (none)
```

diff

```
--- a/grades.py
+++ b/grades.py
@@ -1,4 +1,8 @@
 def average(grades):
-    if len(grades) == 0:
-        return 0
-    return sum(grades) / len(grades)
+    x = 0
+    for g in grades:
+        x = x + g
+    print(x / len(grades))
+    # if len(grades) == 0:
+    #     return 0
```

Decision:_____

Explanation/Justification:

THE ENCORE

```
cd ~/git-katas/squashing
source setup.sh
```

Połącz 5 commitów w jeden poleceniem ***git rebase -i i*** nadaj mu porządny opis. Dlaczego przed PR warto „posprzątać” historię? Dlaczego nie robi się tego na gałęzi, którą ktoś już pobrał?

Check your knowledge

- Czym różni się wynik git merge od git rebase w historii projektu?
- Dlaczego po rebase commity mają nowe hashe?
- Podczas rebase rozwiązałeś konflikt i zrobiłeś git add. Jakie polecenie wykonasz dalej?

  a) git commit b) git merge --continue c) git rebase --continue d) git push
- Podaj złotą zasadę rebase (jednym zdaniem).
- Co to jest Pull Request i po co go się robi, zamiast mergować bezpośrednio do main?
- Wymień 3 rzeczy, które powinien zawierać dobry opis PR.
- Prawda czy fałsz: w code review komentujesz autora, a nie kod.
- Czym różnią się w review statusy Approve i Request changes?